CS 4122: Reinforcement Learning

Programming Assignment #3 (for Module 3)

Release date: 6th April, 2026, 8:15 am (IST)

Due date: 28th April, 2026, 11:59 pm (IST)

Maximum Score: 100 marks (this assignment is graded)

Read the following before you move forward:

1. This programming assignment is worth 10% of the total marks of this course.

2. Late policy: You can be late by a maximum of 3 days. Your assignment will not be accepted more than 3 days after the due date. The actual score and received score (received score is actual score minus penalty for late submission) are related as follows:

$$\text{received score} = \begin{cases} \text{actual score} & ;\text{late by 0 days} \\ 0.9 \cdot \text{actual score} & ;\text{late by 1 day} \\ 0.8 \cdot \text{actual score} & ;\text{late by 2 days} \\ 0.7 \cdot \text{actual score} & ;\text{late by 3 days} \\ 0 & ;\text{late by more than 3 days} \end{cases}$$

3. This is a team project. You have to do the project with the team that was finally selected for you. Only one submission per team. To submit, go to your team's Google drive folder. The Google drive link was already emailed to you. Then go to the sub-folder titled Programming Assignment 3 and drop the following FOUR files (DON'T zip/compress these files, DON'T submit any other files):
   (a) report.pdf (b) GymAirQuality.py (c) train.py (d) Trained policies as .npy files
   Sections 6-8 have **deliverables**. Read these deliverables carefully to understand what needs to be submitted in the report and in the Python codes.

4. The first page of report.pdf should contain a detailed description of the contribution made my individual team members. NOTE: Documenting the work is NOT a valid contribution.

5. Even though each team members may contribute to a specific section of the assignment, every team member should have the technical understanding of the entirety of the project.

6. I will be conducting vivas. Assignment marks will be heavily dependent on these vivas.

7. Plagiarism, if detected, will lead to a score of ZERO for all the team members!
   a. It is ok to take help from AI tools. It is down-right stupid to copy-paste from AI tools and think that is acceptable.
   b. It is ok to take help from other teams. But, it is a bad idea to keep open another teams work in front of you while you do the assignment. While you may think you are just taking help, your reports and your codes may get heavily influenced by the other team to the point that it will qualify as being plagiarized.

Optimizing Air Quality Monitoring using TD-Learning

Section 1: Objectives

The following are the main objectives of this programming assignment:

1. Demonstrate a real-life application of the RL algorithms that you learned in Module 3.
2. Doing one more practice problem where you formulate a problem as a Markov decision process.
3. Coding a custom environment in OpenAI Gymnasium. Coding a custom environment is an essential skill in applied RL. While libraries like Stable Baselines provide ready-made RL agents, it's unlikely you'll find an existing environment that fully matches your application. As a result, you usually need to code a custom environment yourself for your application.
4. Implement a TD-based learning algorithm that you learned in module 3 from scratch.
5. Demonstrate how understanding the mathematics behind an RL algorithm can help in improving the RL algorithm in some special cases.

Section 2: Installation

Installation instructions are same as programming assignment 1. In fact, you don't need any deep learning libraries (like Tensorflow, Keras, and Pytorch) for this programming assignment.

Section 3: Sharing workload

Sharing workload is tricky because the tasks are coupled. The following is my advice:

1. Section 6 is about coding custom environment in OpenAI Gymnasium. You can't fully attempt the later sections without completing section 6; hence the coupling. I suggest that every team member should contribute to section 6. This is also important because of the reason mentioned in section 1.
2. In section 7, there are two tasks related to implementing TD-based RL algorithms. These two tasks are decoupled. You can easily divide these tasks among yourself and do it independently. Also, you really don't have to wait for section 6 to complete before you can start working on section 7. You can still write a skeleton of the code and then fill in the blanks after section 6 is completed.

Section 4: Useful resources

1. In section 6, you will be coding a custom environment. To learn how to code a custom environment, refer to lecture 0, part 4 (there are two video lectures in it containing two examples).
2. In sections 7, you will be training RL agents using temporal difference (TD) based learning algorithms. You can find the pseudocode of these learning algorithms in lectures 25 and 26.
3. Here is a video lecture that demonstrates how to implement expected-SARSA in Python:

   (link to be updated)

Section 5: Problem Setup

You might be aware that air quality in some parts of India, is very poor. Delhi is one of the cities that is often in news in this regard. This is the link to a government website the keeps a log of air quality for different parts of India: https://airquality.cpcb.gov.in/AQI_India/. Better monitoring of air quality is perhaps one of the key steps to improve the air quality. If we know how air quality of different localities of a city evolves over time, we can pin-point the source of pollution and thereby take corrective actions. To do this, we need to install sensors in different parts of the city to measure air quality. This is shown in figure 1. All these sensors measure the air quality in their respective location and communicate it to a central station. To reduce long-term government spending, it is better if these sensors are solar powered.

Figure 1

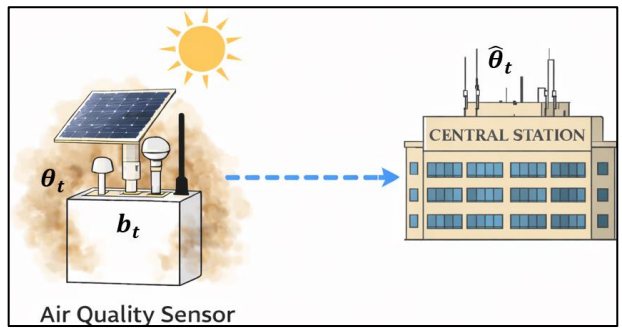

Figure 2

(Figures 1 and 2 are created using ChatGPT)

There are multiple sensors in figure 1. In this programming assignment, we deal with a simpler setup consisting of only one sensor that measures air pollution in its location and transmits it to the central station wirelessly. This is shown in figure 2. Let $\theta_t \in \mathcal{S}$ be the normalized air pollution at time slot t, where $\mathcal{S} = \{0, 0.02, 0.04, \cdots, 0.98, 1\}$. We assume θ_t is a Markov process. $p_{\theta,\theta'}$ is the probability that the air pollution in the current time slot is θ and in the next time slot is θ'. In every time slot, the sensor measures the current air pollution. We assume that the sensor can exactly measure the true air pollution, θ_t.

The sensor is also equipped with a battery. The maximum capacity of the battery is B. Let $b_t \in \{0, 1, 2, \cdots, B\}$ be the battery level at time slot t. The battery is charged using solar energy. Let $\delta_t \in \{0, 1, \cdots, \Delta\}$ be the solar energy in time slot t that charges the battery. δ_t is an iid process. α_δ is the probability that the solar energy is δ. In every time slot, the sensor can choose to transmit to the central station after measuring the air pollution. Each transmission consumes η units from battery. Therefore, the sensor can transmit in time slot t only if $b_t \geq \eta$.

If the sensor transmits, it is not guaranteed that the central station will successfully receive the transmission. This is because wireless channels are noisy. The probability of successful transmission is λ. If the transmission is successful, the central station sends an acknowledgement (ACK) signal back to the sensor. The central station stores the value of the last received transmission from the sensor. Let $\hat{\theta}_t$ denote the value of the last received transmission at time slot t. $\hat{\theta}_t$ can also be interpreted as the central

station's estimate of the air pollution at time slot t. The sensor knows the value of $\hat{\theta}_t$ in time slot t. This is because of the ACK signal. If the sensor receives an ACK signal it know that $\hat{\theta}_t$ is equal to the value it transmitted in the current time slot. If there is no ACK signal, then $\hat{\theta}_t = \hat{\theta}_{t-1}$ implying that there is no update in the estimate. In time slot t, $\hat{\theta}_t$ is decided AFTER the transmission at time slot t. The loss, l_t, of the sensor at time slot t is as follows:

1. $l_t = |\theta_t - \hat{\theta}_t|$ if $\theta_t \leq \hat{\theta}_t$ where $|\cdot|$ is the absolute value operator.
2. $l_t = \mathbf{1.5} \cdot |\theta_t - \hat{\theta}_t|$ if $\theta_t > \hat{\theta}_t$.

Notice the weight 1.5. This implies that there is a higher loss if the estimate is lesser than the true value. So, it is better to estimate that the air pollution is worse than the actual value rather than estimating it is better than the actual value. The objective of the sensor is to minimize the following discounted loss,

$$\sum_{t=0}^{\infty} \beta^t l_t$$

where $\beta \in (0,1)$ is the discount factor. In order to minimize the discounted loss, the sensor has to decide one of the following three actions in time slot t *(the 2nd and the 3rd actions are possible only if $b_t \geq \eta$)*:

Action 1: Not transmit any value.
Action 2: Transmit the true air pollution, θ_t.
Action 3: Transmit the maximum value of air pollution since the last successful transmission. This makes sense because a higher estimate leads to a lower loss compared to lower estimate.

Need for planning?

TD-Learning is part of reinforcement learning and "planning" is an important component of reinforcement learning. So, before going ahead with maths and making things complex, it is a good engineering practice to take a step back and ask a few fundamental questions. From an intuitive point of view, why does the problem setup described above need any sort of planning. More specifically, if there is enough battery to transmit, then why should the sensor simply not take action 2 every time? Why will the sensor even consider taking actions 1 and 3? Let's try to intuitively answer these questions below:

1. **Why take action 1?:** If the true air pollution θ_t is close enough to the central station's estimate $\hat{\theta}_t$, then action 2 will consume battery for a marginal reduction in estimation error $|\theta_t - \hat{\theta}_t|$. Rather, it is better to not transmit in order to save battery for some future time slot when the difference between θ_t and $\hat{\theta}_t$ is significant.
2. **Why take action 3?**: It is intuitive to think that if the sensor decides to transmit, it should transmit the true air pollution so that estimation error is zero. But, this intuition implicitly assumes that we can transmit in every time slot and that transmission will be successful. But, the number of (successful) transmissions may be limited by battery constraints and successful transmission probability λ. If we are not sure whether we can transmit in future time slot (because of battery constraints) and if those transmissions will be successful, it makes sense to transmit a higher value of air pollution (action 3) because the loss is more for under estimation compared to over estimation.

Section 6: Coding Custom Environment (40 marks)

Deliverables:

(15 marks) Formulate the problem setup described in section 5 as an MDP. To do this, by documenting the following in *report.pdf*. VERY IMPORTANT: I want to stress something that was mentioned before. In time slot t, $\hat{\theta}_t$ is decided AFTER the transmission at time slot t.

a. States and state space. *HINT: This problem has four states. Also, in case you did not notice, $\mathcal{S}$ is a discrete set. Hence, the state space of θ_t is discrete.*
b. Action and action space.
c. Reward.
d. State transition equations.

2. (25 marks) Using the answer to the previous question, code a custom OpenAI Gymnasium environment for the problem setup described in section 5. The skeleton of this environment is there in *GymAirQuality.py* that is provided to you. *GymAirQuality.py* contains a class named *SensorTransmissionEnv*. Implement the following functions of *SensorTransmissionEnv*:
   a. *__init()__:* This function should initialize all the system parameters. The system parameters MUST be initialized as follows:
      - Set $\lambda = 0.1$, $B = 10$, $\eta = 2$, and $\Delta = 3$.
      - Recall that θ_t is a Markov process. If we consider the probabilities, $p_{\theta,\theta'}$, for all θ and θ', it forms the Markovian matrix. You are given a file *air.npy* that contains this Markovian matrix that you must use for this assignment. The $(i,j)^{th}$ element of this Numpy matrix is the probability $p_{\theta,\theta'}$ where $\theta = 0.02i$ and $\theta' = 0.02j$.
      - Recall that δ_t is an iid process. If we consider the probabilities, α_δ, for all δ, it forms a vector. You are given a file *solar.npy* that contains this vector that you must use for this assignment. The ith element of this Numpy vector is the probability α_δ where $\delta = i$.

      You MUST also define the observation space (similar to state space) and action space in this function. Note that for this problem action space is a function of states. But, according to my knowledge, OpenAI Gymnasium has no option to define a state dependent action space. So, while defining the action space in this function, you can assume that all actions are possible for every state. But then, when you are coding the RL agent in the next section you must ensure that the actions chosen by the RL agent respects the true state-dependent action space.
   b. *reset():* The function to reset the environment in the beginning of every episode. There are four states for this problem. Three of these states must be set uniformly at random in its respective state space. Setting the fourth state is a little tricky and I leave that upto you.
   c. *step():* This function should take the action as input and return the following:
      i. Next state.
      ii. Current reward.
      iii. $terminated$. This should be $False$.
      iv. $truncated$. This setup is a continuous task. But, truncate an episode after 288 time slots, i.e. each episode is one day with the duration of one time slot being $5\ min$.
      v. $info$. This should be an empty dictionary.

Section 7: Training RL algorithms (45 marks)

In this section, you will be coding two temporal difference (TD) based learning algorithms for the environment that you coded in section 6. You must follow these points while coding these two algorithms:

1. You are given a Python script *train.py* which already contains the skeleton of the code where the number of episodes, discount factor, and the learning rate are already set. The Python script also saves the policy that you learn as .npy files. You MUST NOT TAMPER with this skeleton. Just implement the two functions *QLearning()* and *QLearning_StructuralKnowledge()*. Read the deliverables below to understand what needs to be implemented in these two functions. Any additional functions required to implement these two functions must be declared inside *train.py*.
2. Use discount factor $\beta = 0.98$ and learning rate $\alpha = 0.1$. It is your task to decide the number of episodes to train and the schedule of the exploration probability.
3. Testing policy: You must be testing the policy as mentioned in the YouTube video link below:

   https://youtu.be/_XxrN5-dvtY

Deliverables:

1. (15 marks) Implement QLearning for the custom environment that you coded in the previous section. You must implement this code in the function *QLearning()* of the Python script *train.py* that is provided to you. The following instructions must be followed or else you get ZERO for this question:
   - You must be testing the policy as mentioned in the beginning of this section. Save this testing data as it will be useful in the next section.
   - For this problem, action space is a function of the state. Your must account for this.
   - *QLearning()* should return the optimal policy (NOT the Q-value) as a Numpy array. Submit the final policy that you learned as *policy1.npy*.

2. (30 marks) Watch the following YouTube video that discusses a TD-learning algorithm that you have to implement (both link discusses the same algorithm; watch video 1 first):

   Video 1: *https://youtu.be/oi6QSBfwQZA*
   Video 2: *https://youtu.be/Wj6i7r3AGPc*

   After watching the videos, you must do the following:
   - Video 2 discusses multiple cases. Some of the cases are discussed in details in the video while others are left for to complete. Neatly document all these cases in *report.pdf*.
   - Implement the TD learning mentioned in the video for the custom environment that you coded in the previous section. You must implement this code in the function *QLearning_StructuralKnowledge()* of the Python script *train.py* that is provided to you. The following instructions must be followed or else you get ZERO for the code:
     - IMPORTANT: You code should only consider those cases that you document in *report.pdf*.
     - You must be testing the policy as mentioned in the beginning of this section.
     - For this problem, action space is a function of the state. Your must account for this.
     - *QLearning_StructuralKnowledge()* should return the optimal policy (NOT the Q-value) as a Numpy array. Submit the final policy that you learned as *policy2.npy*.

Section 8: Analysis (15 marks)

Deliverables:

1. (5 marks) You must have tested both the TD-based learning algorithms during training as mentioned in the beginning of section 7. In the process of doing so, you must have saved the average of the sum of reward. Make a plot of the average of the sum of reward vs episode for both the algorithm and included it in *report.pdf*. Both the plots should be on the same graph. Which of the two algorithms is learning faster?

You can answer the following two questions for any one of the policies that you learned in the previous section. You can do whatever analysis you want. But, you must explain your analysis in details in *report.pdf*. Whatever you explain must have proof in form of plots, tables etc.

2. (5 marks) Is there a certain pattern of state for which action 1 is the optimal action?

3. (5 marks) Is there a certain pattern of state for which action 3 is the optimal action?